

Assembly Programming for Arduino Uno R3 (ATmega328P)

Dr. Hiran Ekanayake

University of Colombo School of Computing

2026

Assembly Programming

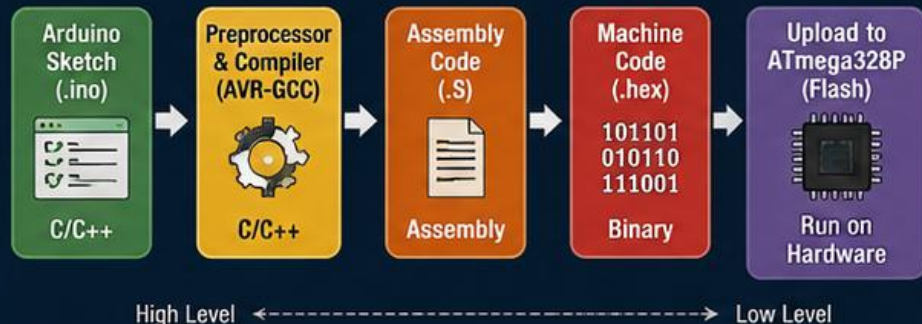
- The **Arduino Uno R3** is based on the ATmega328P, an **8-bit AVR RISC microcontroller** running at **16 MHz**. Although most Arduino programs are written in C/C++, every Arduino sketch is ultimately compiled into **AVR machine code**, making assembly language the lowest programming level available on the board.
- Assembly programming is useful when you need:
 - Maximum execution speed
 - Minimum program size
 - Precise timing (e.g., generating communication protocols)
 - Understanding how the microcontroller works internally
 - Developing bootloaders, device drivers, or embedded operating systems

FROM SKETCH TO SILICON LOW LEVEL ARDUINO PROGRAMMING

Understand the journey from your Arduino sketch to machine code and how the ATmega328P really works!



1. PROGRAM COMPILEATION FLOW



2. PROGRAMMING LEVELS

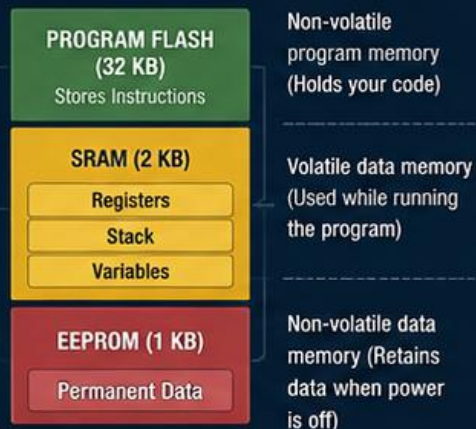


3. ARDUINO UNO R3 (ATmega328P)



CPU	: 8-bit AVR RISC
Clock	: 16 MHz
Flash	: 32 KB
SRAM	: 2 KB
EEPROM	: 1 KB
I/O Pins	: 23
Timers	: 3
10-bit ADC	: 6 ch
USART / SPI / I ² C	: Yes

4. AVR MEMORY ORGANIZATION



5. ASSEMBLY CODE EXAMPLE (LED BLINK ON PIN 13)

```
.include "m228pdef.inc" ; Device definitions
.org 0x0000 ; Reset vector
rjmp RESET
RESET:
    sbi DDRB,5 ; Set PB5 (D13) as OUTPUT
LOOP:
    sbi PORTB,5 ; LED ON
    rcall DELAY ; Call delay
    cbi PORTB,5 ; LED OFF
    rcall DELAY ; Call delay
    rjmp LOOP ; Repeat forever
;----- Delay Subroutine -----
DELAY:
    ldi r18,255
    ldi r19,255
L1:
    dec r19
    brne L2
    dec r18
    brne L1
    ret
```

- What it does:
- Configures PB5 (D13) as output
 - Turns LED ON
 - Waits (delay loop)
 - Turns LED OFF
 - Repeats forever

6. LOW LEVEL I/O – WORKING WITH REGISTERS

Arduino (High Level)	Assembly (Low Level)
<code>Cinfigrle(13, OUTPUT);</code>	<code>SBI DDRB,5 ; Set PB5 as OUTPUT</code>
<code>digitalWrite(13, HIGH);</code>	<code>SBI PORTB,5 ; Set PB5 HIGH (LED ON)</code>
<code>digitalWrite(13, LOW);</code>	<code>CB1 PORTB,5 ; Set PB5 LOW (LED OFF)</code>

Arduino Pin to ATmega328P Mapping (Selected)

Arduino Pin	0 (RX)	1 (TX)	2	3	4	5	6	7	8	9	10	11	12	13 (LED)
ATmega328P	PD0	PD1	PD2	PD3	PD4	PD5	PD6	PD7	PB0	PB1	PB2	PB3	PB4	PB5
Port	PORTD (Digital 0-7)							PORTB (Digital 8-13)						

7. COMMON AVR INSTRUCTIONS

Instruction	Meaning
<code>LDI Rd, K</code>	Load Immediate value K into register Rd
<code>MOV Rd, Rr</code>	Copy register Rr to Rd
<code>ADD Rd, Rr</code>	Add Rr to Rd
<code>SUB Rd, Rr</code>	Subtract Rr from Rd
<code>INC Rd</code>	Increment Rd
<code>DEC Rd</code>	Decrement Rd
<code>SBI I/O, b</code>	Set Bit b in I/O register
<code>CB1 I/O, b</code>	Clear Bit b in I/O register
<code>RJMP label</code>	Relative Jump
<code>CALL label</code>	Call subroutine
<code>RET</code>	Return from subroutine

Why Go Low Level?

- ✓ Maximum speed
- ✓ Minimal code size
- ✓ Precise timing
- ✓ Direct hardware control
- ✓ Better understanding of how Arduino works

Tools You Can Use

- Arduino IDE (with AVR-GCC)
- Microchip Studio (Atmel Studio)
- AVR-GCC + avr-as (Command Line)
- Programmer (USBasp / Atmel-ICE)

8. HOW IT WORKS – BEHIND THE SCENES



9. EXAMPLE: READ A DIGITAL INPUT (BUTTON ON D2)

Arduino (High Level)	Assembly (Low Level)
<code>int state = digitalRead(2);</code>	<code>IN r16, PIND ; Read PORTD</code> <code>ANDI r16, 0x04 ; Mask PD2 (D2)</code> <code>; Result in r16 (0 or 1)</code>

10. TAKEAWAY

- High level languages make development easy.
- Assembly language gives you power, speed and control.
- Knowing both makes you a better embedded developer!

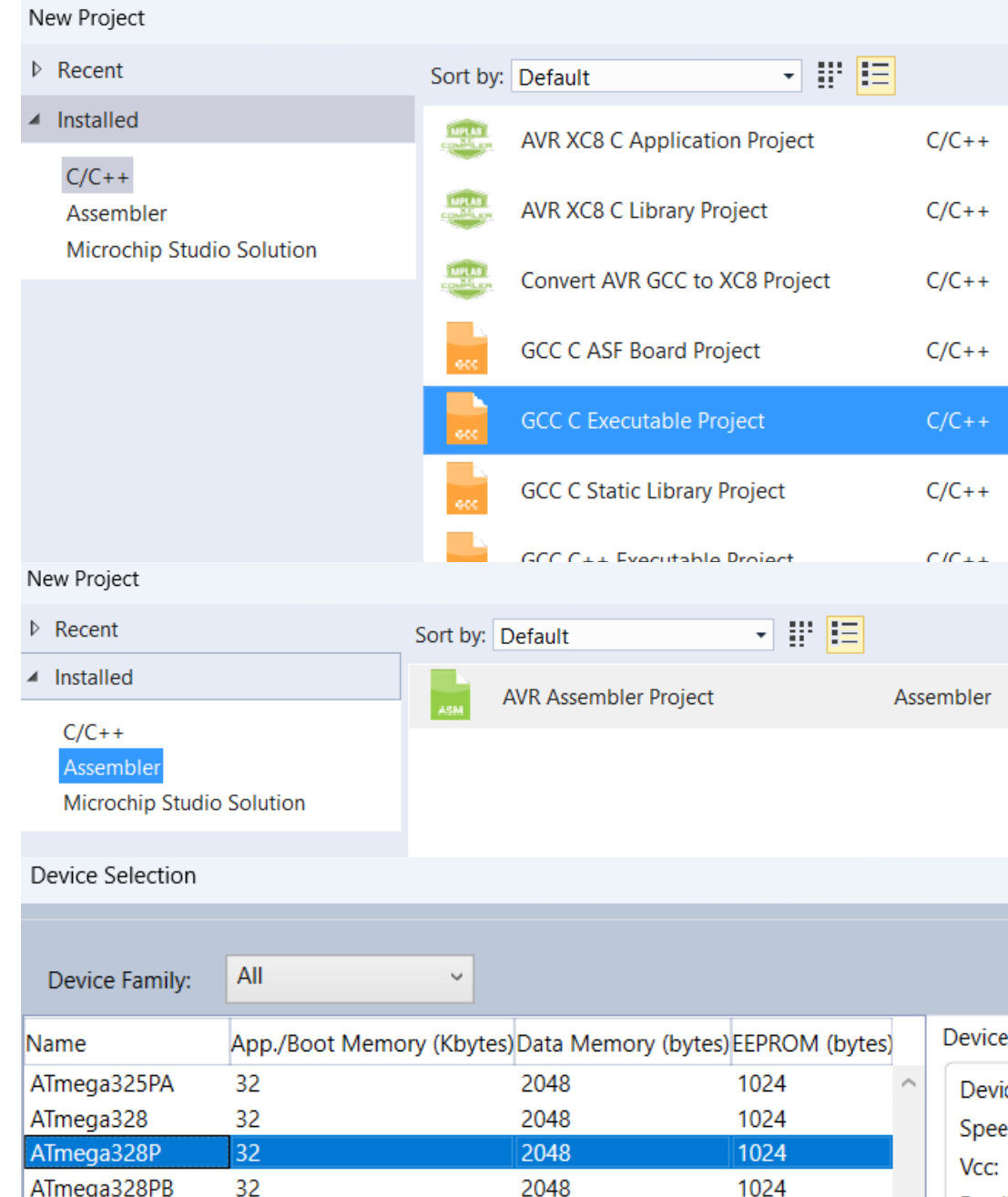
Steps

- **Step 1. Install Software**

- [Microchip Studio for AVR and SAM Devices \(v7.0.2594 2022\)](#)
- **Arduino IDE** (needed to install USB drivers and the bootloader if not already present).
- During installation, Microchip Studio installs: AVR Assembler, AVR Linker, AVR GCC, AVRDUDE (or compatible programming tools).

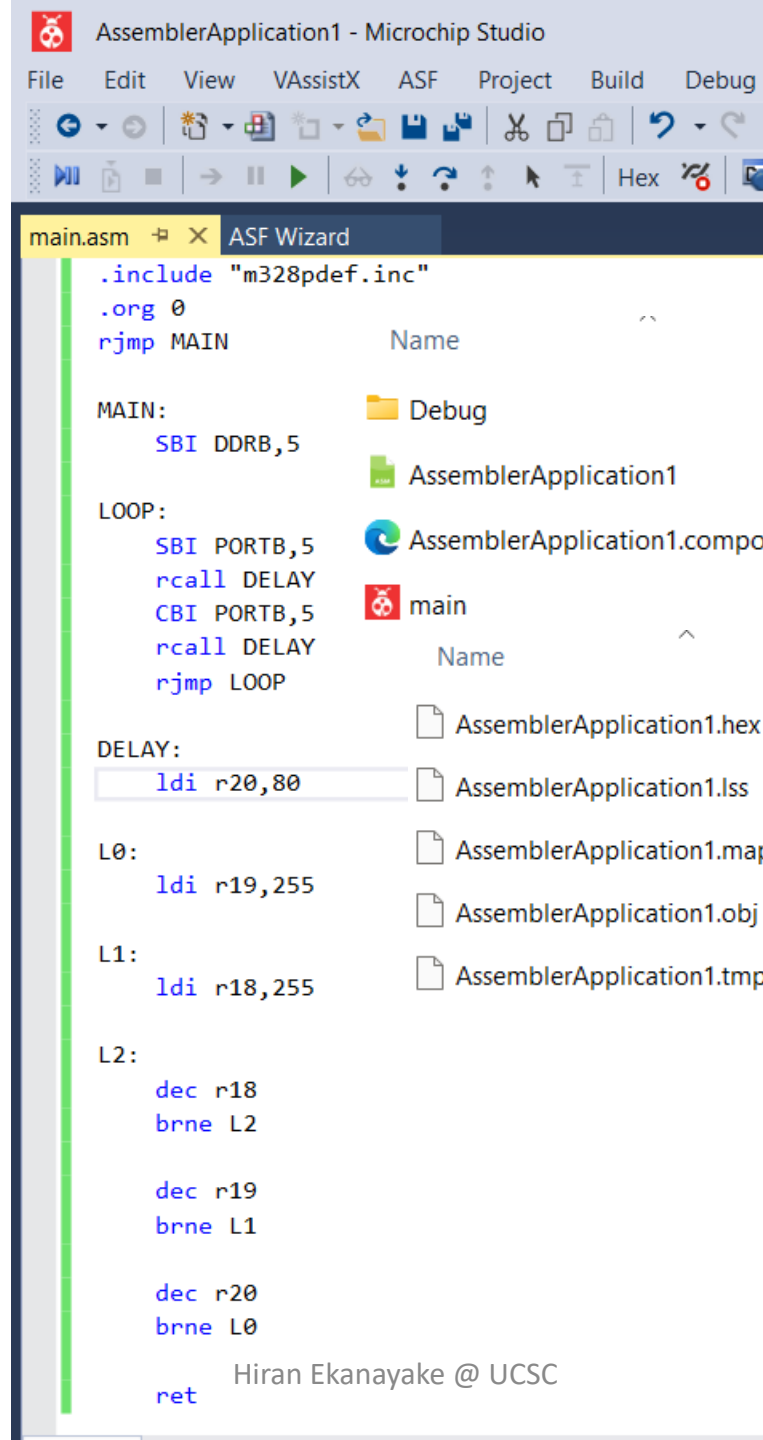
- **Step 2. Create an Assembly Project**

- File → New → Project
- Choose: *GCC C Executable Project* or *Assembler Project*
- Device: *ATmega328P*



Steps Cont.

- **Step 3. Write Assembly Code**
- **Step 4. Build**
 - Press Build or F7
 - Output: AssemblerApplication1.hex



The screenshot shows the Microchip Studio IDE. The main window displays the assembly code for `main.asm`. The code includes a header file `"m328pdef.inc"` and defines a `MAIN` routine. The `MAIN` routine starts with `SBI DDRB, 5` and enters a `LOOP` containing `SBI PORTB, 5`, `rcall DELAY`, `CBI PORTB, 5`, `rcall DELAY`, and `rjmp LOOP`. The `DELAY` routine is defined with `ldi r20, 80`. There are also labels `L0`, `L1`, and `L2` with instructions `ldi r19, 255`, `ldi r18, 255`, `dec r18`, `brne L2`, `dec r19`, `brne L1`, `dec r20`, `brne L0`, and `ret`. The project explorer on the right shows the project structure, including the `main` file and the output files `AssemblerApplication1.hex`, `AssemblerApplication1.lss`, `AssemblerApplication1.map`, `AssemblerApplication1.obj`, and `AssemblerApplication1.tmp`.

```
.include "m328pdef.inc"
.org 0
rjmp MAIN

MAIN:
    SBI DDRB, 5

LOOP:
    SBI PORTB, 5
    rcall DELAY
    CBI PORTB, 5
    rcall DELAY
    rjmp LOOP

DELAY:
    ldi r20, 80

L0:
    ldi r19, 255

L1:
    ldi r18, 255

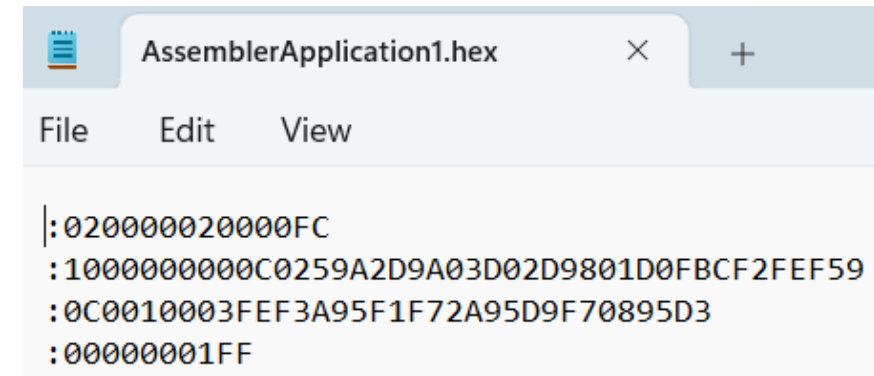
L2:
    dec r18
    brne L2

    dec r19
    brne L1

    dec r20
    brne L0

    ret
```

Name	Date modified	Type	Size
Debug	7/12/2026 10:16 AM	File folder	
AssemblerApplication1	7/12/2026 10:12 AM	Assembler Project	3 KB
AssemblerApplication1.componentinfo	7/12/2026 10:12 AM	Microsoft Edge HT...	3 KB
main	7/12/2026 10:14 AM	ASM File	1 KB
Name	Date modified	Type	Size
AssemblerApplication1.hex	7/12/2026 10:16 AM	HEX File	1 KB
AssemblerApplication1.lss	7/12/2026 10:16 AM	LSS File	74 KB
AssemblerApplication1.map	7/12/2026 10:16 AM	MAP File	20 KB
AssemblerApplication1.obj	7/12/2026 10:16 AM	OBJ File	1 KB
AssemblerApplication1.tmp	7/12/2026 10:16 AM	TMP File	2 KB



The screenshot shows the `AssemblerApplication1.hex` file open in the Microchip Studio IDE. The file contains a list of hexadecimal values, likely representing the compiled assembly code.

```
:0200000020000FC
:1000000000C0259A2D9A03D02D9801D0FBCF2FEF59
:0C0010003FEF3A95F1F72A95D9F70895D3
:00000001FF
```

Steps Cont.

- **Step 5. Connect Arduino**

- Identify the COM port, e.g. COM6 (can see from Windows Device Manager or Arduino IDE)

- **Step 6. Upload**

- Open the PowerShell (or Command Prompt)
- Navigate to the folder where avrdude.exe is located
 - AVRDUDE can be located under the Arduino installation folder, e.g.,
C:\Users\XXX\AppData\Local\Arduino15\packages\arduino\tools\avrdude\8.0.0-arduino1\bin
- Copy the AssemblerApplication1.hex file to this location.
- In PowerShell: `.\avrdude.exe -C "..\etc\avrdude.conf" -p atmega328p -c arduino -P COM6 -b 115200 -D -U flash:w:AssemblerApplication1.hex:i`
- Note: If you don't want to type `.\` or the full path every time, you can add the AVRDUDE bin directory to your Windows PATH environment variable.

```
PS C:\Users\HBE\AppData\Local\Arduino15\packages\arduino\tools\avrdude\8.0.0-arduino1\bin> avrdude.exe -C "..\
etc\avrdude.conf" -p atmega328p -c arduino -P COM6 -b 115200 -D -U flash:w:AssemblerApplication1.hex:i
Reading 34 bytes for flash from input file AssemblerApplication1.hex
Writing 34 bytes to flash
Writing | ##### | 100% 0.03 s
Reading | ##### | 100% 0.02 s
34 bytes of flash verified

Avrdude done. Thank you.
```

Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

✓ → ⚙

Arduino UNO

📶 🔍

Blink.ino

24
25 // the setup function runs once when you press reset or power the board
26 void setup() {
27 // initialize digital pin LED_BUILTIN as an output.
28 pinMode(LED_BUILTIN, OUTPUT);
29 }
30
31 // the loop function runs over and over again forever
32 void loop() {
33 digitalWrite(LED_BUILTIN, HIGH); // change state of the LED by setting the pin to the HIGH voltage level
34 delay(1000); // wait for a second
35 digitalWrite(LED_BUILTIN, LOW); // change state of the LED by setting the pin to the LOW voltage level
36 delay(1000); // wait for a second
37 }
38

Output

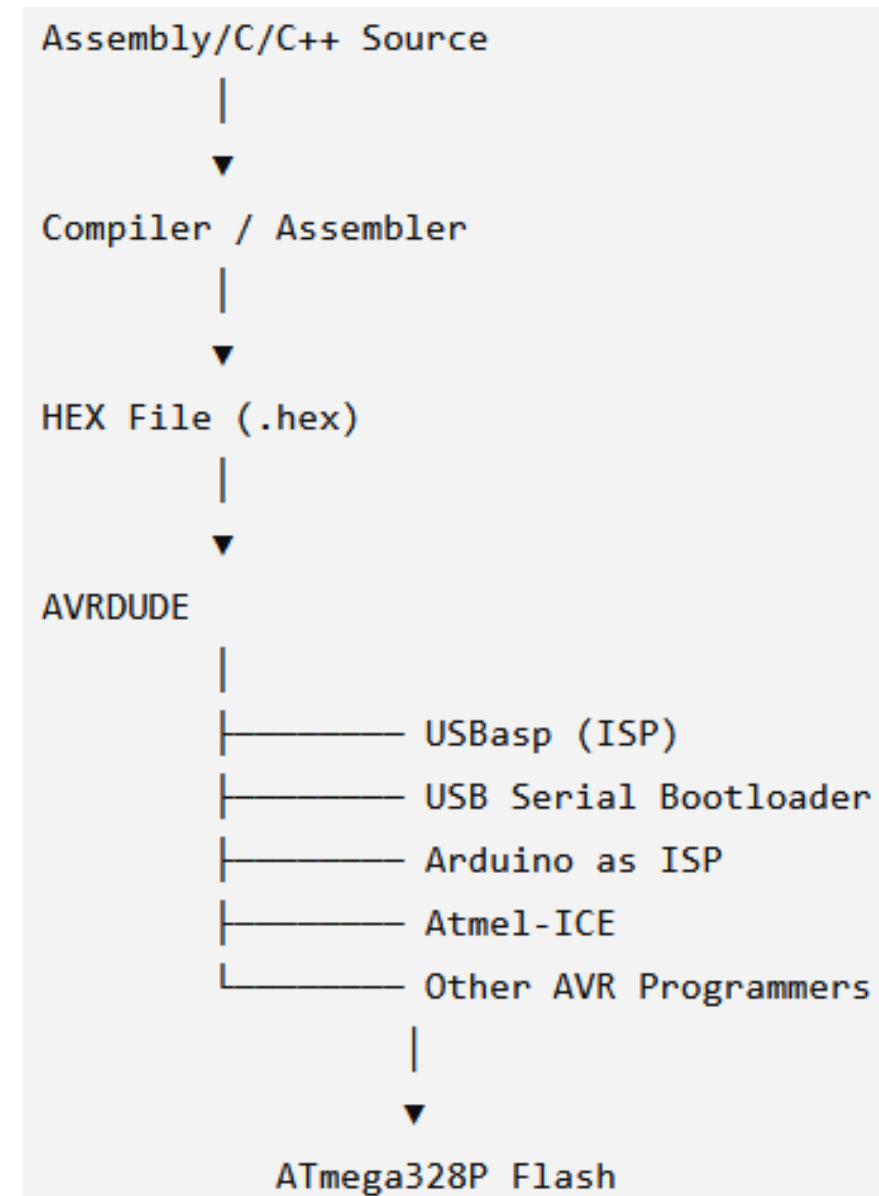
Sketch uses 924 bytes (2%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048 bytes.

34 bytes vs 924 bytes!

Ln 1, Col 1 Arduino UNO on COM6 🔔 2 📄

What is AVRDUDE?

- **AVRDUDE (AVR Downloader/UploaDEr)** is the standard command-line utility used to **program AVR microcontrollers**. It transfers compiled firmware (typically an Intel HEX file) from your computer into the Flash memory of AVR devices such as the **ATmega328P** used on the Arduino Uno R3.
- Whenever you click **Upload** in the Arduino IDE or **Burn Bootloader**, the IDE usually invokes **AVRDUDE** behind the scenes.
- AVRDUDE is responsible only for **communicating with the programmer or bootloader**. It does not compile your source code.



What Can AVRDUDE Do?

- Program Flash memory
- Read Flash memory
- Erase Flash memory
- Read/write EEPROM
- Read/write Fuse bits
- Read/write Lock bits
- Verify programmed data
- Read the device signature
- Burn bootloaders

Basic Command Structure

- **Example 1: Detect the Microcontroller**

- `avrdude -c usbasp -p m328p`
- Typical output: Device signature = 0x1E950F

- **Example 2: Upload a Program**

- `avrdude -c usbasp -p m328p -U flash:w:blink.hex`
- `avrdude -c arduino -p m328p -P COM6 -b115200 -U flash:w:Blink.hex`
(Normal Arduino IDE uploads)

- **Example 3: Read Flash Memory**

- `avrdude -c usbasp -p m328p -U flash:r:backup.hex:l`

- **Example 4: Read EEPROM**

- `avrdude -c usbasp -p m328p -U eeprom:r:data.hex:l`

- **Example 5: Read Fuse Bits**

- `avrdude -c usbasp -p m328p -U lfuse:r:-:h`

- **Example 6: Program the Bootloader**

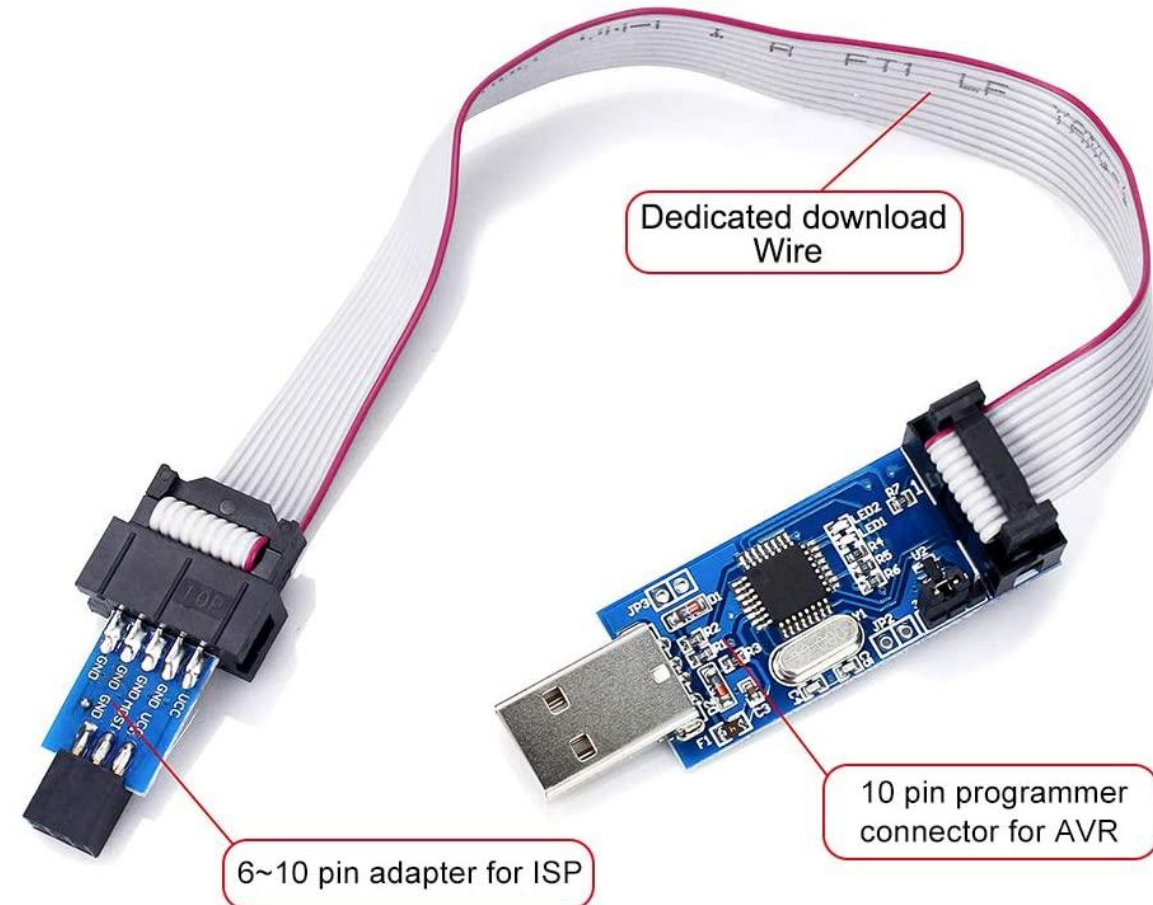
- `avrdude -c usbasp -p m328p -U flash:w:optiboot.hex`

Option	Meaning
-c	Programmer
-p	Microcontroller
-P	Serial port (for bootloader uploads)
-b	Baud rate
-U	Memory operation
-v	Verbose output
-e	Chip erase
-F	Ignore signature check (use with caution)

USBasp Programmer

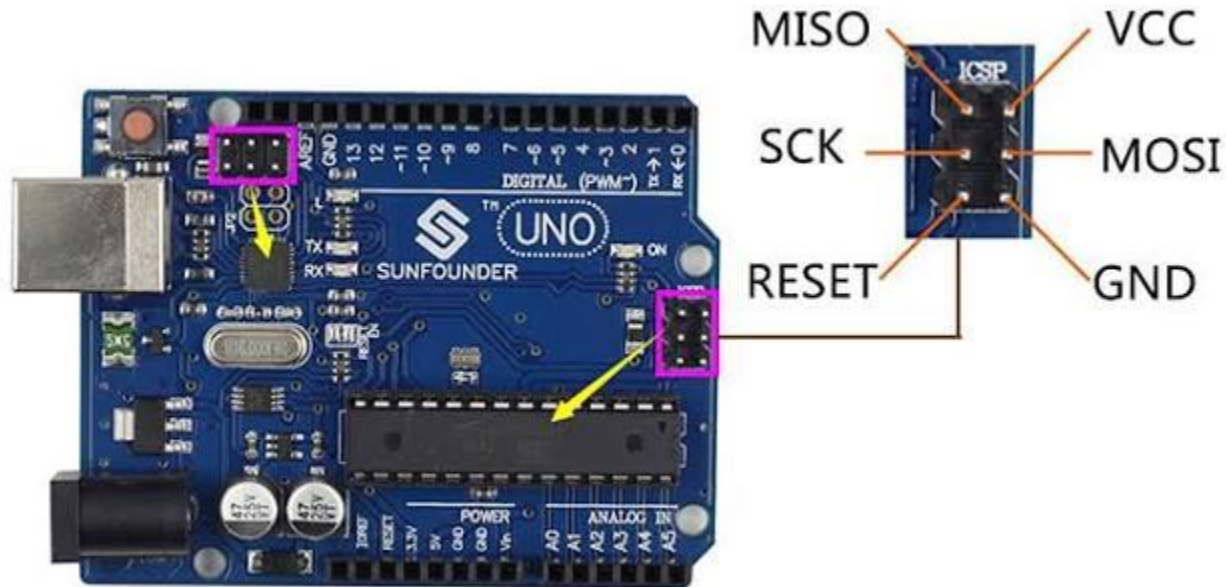
What is USBasp?

- **USBasp** is an inexpensive USB-based **In-System Programmer (ISP)** for **AVR microcontrollers**. It allows a computer to program the flash memory, EEPROM, and fuse bits of AVR chips directly through the **ISP (In-System Programming)** interface, without relying on a bootloader.
- It was originally developed by Thomas Fischl and has become one of the most widely used programmers for AVR development.



Arduino Uno ICSP Header

- USBasp connects directly to this header.



USBasp Pin	ATmega328P Function
MOSI	Master Out Slave In
MISO	Master In Slave Out
SCK	Serial Clock
RESET	Reset target MCU
VCC	Power (optional)
GND	Ground

Why use USBasp instead of the Arduino bootloader?

Arduino Bootloader	USBasp
Uses USB Serial	Uses SPI (ISP)
Requires bootloader	No bootloader required
Cannot recover a corrupted bootloader	Can install or restore the bootloader
Cannot easily program fuse bits	Can read/write fuse bits
Slower	Faster
Loses about 0.5 KB of Flash to bootloader	Entire Flash available
Upload via COM port	Upload directly through USBasp

Steps

- **Install the Driver for USBasp**

- Some USBasp clones require a driver installation.
- Plug in USBasp and install the driver using Zadig (<https://zadig.akeo.ie/>)

- **Programming with AVRDUDE**

- In PowerShell: `avrdude -c usbasp -p m328p -U flash:w:AssemblerApplication1.hex`

```
Reading 34 bytes for flash from input file AssemblerApplication1.hex
Writing 34 bytes to flash
Writing | ##### | 100% 0.15 s
Reading | ##### | 100% 0.07 s
34 bytes of flash verified

Avrdude done. Thank you.
```

Understand the Program

- **Include File**

- This file contains definitions for the ATmega328P, such as: DDRB PORTB PINB TCCR1A TCCR1B SPH SPL
- Without it, you would need to use raw I/O register addresses such as: SBI 0x04,5

- **Program Origin**

- Here it tells the assembler to place the next instruction at Flash address **0x0000**.
- Since the AVR begins execution at address 0 after reset, this is where the reset vector belongs.

```
.include "m328pdef.inc"

.org 0
    rjmp RESET

RESET:
    sbi DDRB,5           ; PB5 (Arduino D13) as output

LOOP:
    sbi PORTB,5          ; LED ON
    rcall DELAY

    cbi PORTB,5          ; LED OFF
    rcall DELAY

    rjmp LOOP

DELAY:
    ldi r20,80

L0:
    ldi r19,255

L1:
    ldi r18,255

L2:
    dec r18
    brne L2

    dec r19
    brne L1

    dec r20
    brne L0

    ret
```

Understand the Program Cont.

- **Reset Vector**

- RJMP means Relative Jump.
- After reset: Power ON → Address 0 → RJMP RESET → RESET:
- This transfers execution to the RESET label.

- **Configure the LED Pin**

- SBI means Set Bit in I/O Register.
- DDRB is the Data Direction Register for Port B.
- Bit 5 is now 1, making PB5 (Arduino D13) an output.
- Equivalent Arduino code: pinMode(13, OUTPUT);

```
.include "m328pdef.inc"

.org 0
    rjmp RESET

RESET:
    sbi DDRB,5           ; PB5 (Arduino D13) as output

LOOP:
    sbi PORTB,5          ; LED ON
    rcall DELAY

    cbi PORTB,5          ; LED OFF
    rcall DELAY

    rjmp LOOP

DELAY:
    ldi r20,80

L0:
    ldi r19,255

L1:
    ldi r18,255

L2:
    dec r18
    brne L2

    dec r19
    brne L1

    dec r20
    brne L0

    ret
```

Understand the Program Cont.

- **Main Loop**

- LOOP: - This label marks the beginning of the infinite loop.
- sbi PORTB,5 - This sets PB5 HIGH. Equivalent Arduino code: digitalWrite(13, HIGH);
- rcall DELAY - Call the Delay. Before jumping, the CPU pushes the return address onto the stack so it knows where to continue after RET.
- cbi PORTB,5 - CBI means Clear Bit in I/O Register. It clears bit 5 of PORTB. Equivalent Arduino code: digitalWrite(13, LOW);
- rjmp LOOP - This jumps back to the LOOP label forever.

```
.include "m328pdef.inc"

.org 0
    rjmp RESET

RESET:
    sbi DDRB,5           ; PB5 (Arduino D13) as output

LOOP:
    sbi PORTB,5          ; LED ON
    rcall DELAY

    cbi PORTB,5          ; LED OFF
    rcall DELAY

    rjmp LOOP

DELAY:
    ldi r20,80

L0:
    ldi r19,255

L1:
    ldi r18,255

L2:
    dec r18
    brne L2

    dec r19
    brne L1

    dec r20
    brne L0

    ret
```

Understand the Program Cont.

- **The Delay Routine**

- This is the heart of the timing.
- `ldi r20,80` – LDI means Load Immediate. Register R20 is loaded with 80. This controls how many times the middle loop executes.
- `dec r18` – Decrement the content of r18 by 1.
- `brne L2` - BRNE means Branch if Not Equal to Zero. If R18 is not zero, jump back to L2. When it reaches zero, continue to next instruction. So the inner loop runs **255 iterations**.
- The middle loop executes **255 inner loops**.
- The entire middle loop executes **80 times**.
- Total Loop Iterations = $80 \times 255 \times 255 = 5,202,000$.

- **Returning from the Subroutine**

- `RET` pops the saved return address from the stack and resumes execution immediately after the `RCALL DELAY` instruction.

```
.include "m328pdef.inc"

.org 0
    rjmp RESET

RESET:
    sbi DDRB,5           ; PB5 (Arduino D13) as output

LOOP:
    sbi PORTB,5          ; LED ON
    rcall DELAY

    cbi PORTB,5          ; LED OFF
    rcall DELAY

    rjmp LOOP

DELAY:
    ldi r20,80

L0:
    ldi r19,255

L1:
    ldi r18,255

L2:
    dec r18
    brne L2

    dec r19
    brne L1

    dec r20
    brne L0

    ret
```


AVR ASSEMBLY PROGRAM (ATmega328P) – LED BLINK WITH 3-LEVEL NESTED DELAY

This program turns the onboard LED (D13 / PB5) ON and OFF forever using a software delay created with three nested loops.



1. ASSEMBLY SOURCE CODE

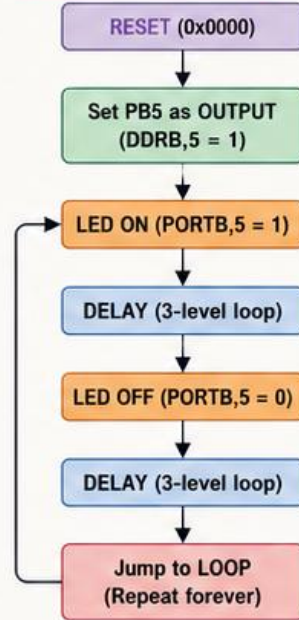
```
.include "m328pdef.inc" ; Device definitions
.org 0 ; Reset vector at address 0x0000
rjmp RESET ; Jump to start of program

RESET:
sbi DDRB, 5 ; Make PB5 (D13) as output

LOOP:
sbi PORTB, 5 ; LED ON
rcall DELAY ; Delay
cbi PORTB, 5 ; LED OFF
rcall DELAY ; Delay
rjmp LOOP ; Repeat forever

DELAY:
ldi r20, 80 ; Outer loop counter
L0:
ldi r19, 255 ; Middle loop counter
L1:
ldi r18, 255 ; Inner loop counter
L2: dec r18 ; Decrement inner counter
brne L2 ; If not zero, repeat L2
dec r19 ; Decrement middle counter
brne L1 ; If not zero, repeat L1
dec r20 ; Decrement outer counter
brne L0 ; If not zero, repeat L0
ret ; Return from subroutine
```

2. PROGRAM FLOW



3. FLASH MEMORY MAP (32 KB)

Address	Machine Code (16-bit)	Assembly Instruction	Meaning
0x0000	C000	RJMP RESET	Start of program after reset
0x0002	9A25	SBI DDRB, 5	PB5 (D13) as output
0x0004	9A2D	SBI PORTB, 5	LED ON
0x0006	D003	RCALL DELAY	Call delay
0x0008	982D	CBI PORTB, 5	LED OFF
0x000A	D001	RCALL DELAY	Call delay
0x000C	CFFB	RJMP LOOP	Repeat forever
0x000E	E540	LDI R20, 80	Outer loop = 80
0x0010	EF3F	LDI R19, 255	Middle loop = 255
0x0012	EF2F	LDI R18, 255	Inner loop = 255
0x0014	952A	DEC R18	Inner loop: decrement
0x0016	F7F1	BRNE L2	If R18 != 0, repeat L2
0x0018	953A	DEC R19	Middle loop: decrement
0x001A	F7D9	BRNE L1	If R19 != 0, repeat L1
0x001C	954A	DEC R20	Outer loop: decrement
0x001E	F7C1	BRNE L0	If R20 != 0, repeat L0
0x0020	9508	RET	Return to caller

Program size = 34 bytes (0x22)
Only 0.10% of the 32 KB Flash is used!

Flash Memory Overview



Key Points

- ✓ CPU always starts from address 0x0000.
- ✓ RJMP RESET jumps to the start of code.
- ✓ PB5 is controlled through DDRB and PORTB.
- ✓ RCALL pushes return address to stack. RET pops it and returns.
- ✓ BRNE = Branch if Not Equal to Zero.
- ✓ Three nested loops create a long software delay.

Delay Calculation (approx.)

Iterations = $80 \times 255 \times 255$
= 5,202,000 inner loop cycles
At 16 MHz, this gives a delay of hundreds of milliseconds, so the LED blinks visibly.

4. HEX FILE EXPLAINED (Intel HEX Format)

```
:02000000200000FC ; Extended Segment Address (ignored here)
:1000000000C0259A2D9A03D02D9801D0FBCF40E552 ; 16 bytes starting at 0x0000
:100010003FEF2FEF2A95F1F73A95D9F74A95C1F7B7 ; 16 bytes starting at 0x0010
:02002000089541 ; 2 bytes starting at 0x0020
:00000001FF ; End of File
```

Intel HEX Record Format: `:llaaaa[dd... ]cc`

ll Length	aaaa Address	tt Record Type	dd... Data Bytes	cc Checksum
Number of data bytes (1 byte)	Start address (2 bytes)	00 = Data 02 = Ext. Segment Addr 01 = End of File	Actual data (n bytes)	Two's complement checksum (1 byte)

5. INSTRUCTION OVERVIEW

RJMP k	Relative jump to address within $\pm 2K$ words.
SBI I, b	Set bit b in I/O register I.
CBI I, b	Clear bit b in I/O register I.
LDI Rd, K	Load immediate value K into register Rd.
RCALL k	Call subroutine (push return address).
RET	Return from subroutine (pop address).
DEC Rd	Decrement register Rd.
BRNE k	Branch to k if Rd $\neq$ 0.

6. REGISTERS USED

r20 – Outer loop counter
Loaded with 80 (0x50)

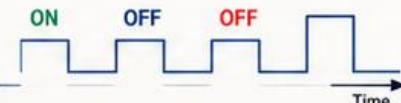
r19 – Middle loop counter
Loaded with 255 (0xFF)

r18 – Inner loop counter
Loaded with 255 (0xFF)

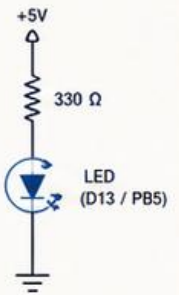
All registers are general purpose 8-bit registers in ATmega328P.

7. PROGRAM BEHAVIOR

- ✓ After reset, PB5 is configured as output.
- ✓ LED (D13) is turned ON.
- ✓ The delay subroutine is called.
- ✓ LED (D13) is turned OFF.
- ✓ Delay again.
- ✓ The loop repeats forever.



8. HARDWARE USED



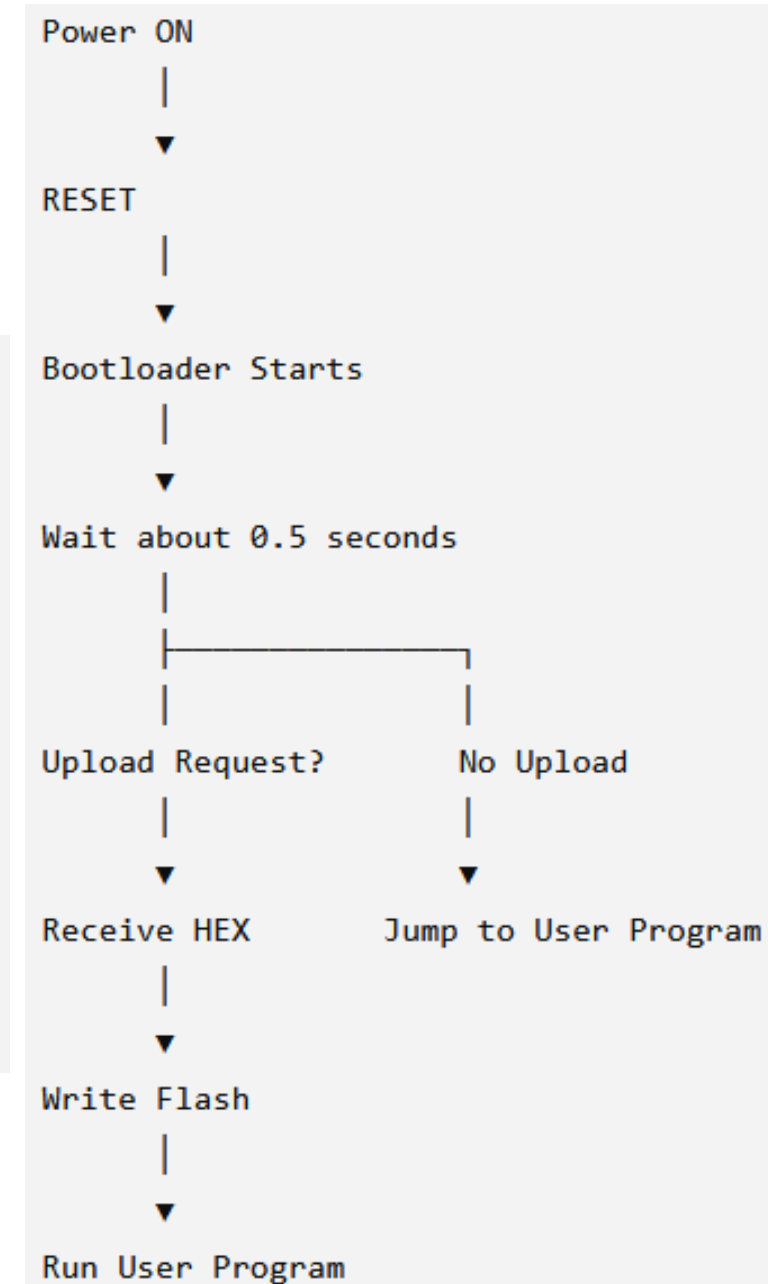
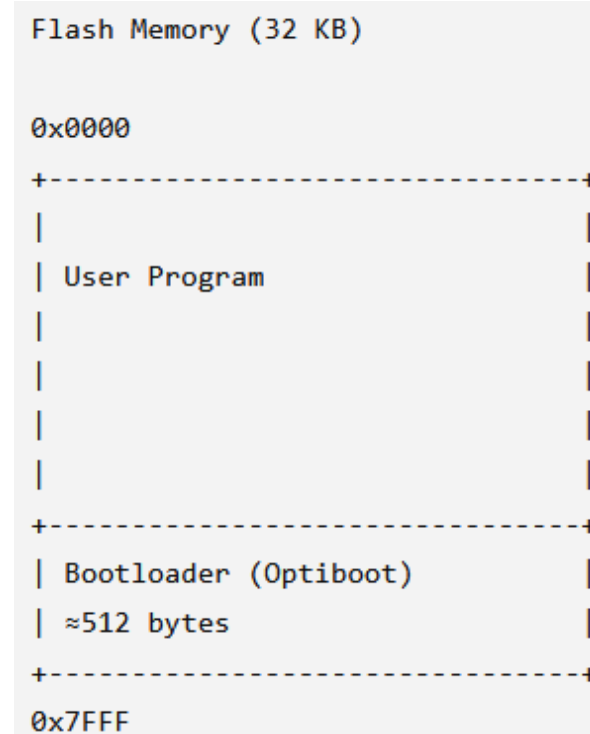
This small program demonstrates the power of AVR assembly: direct hardware control, efficient code, and full understanding of what the CPU executes!

Arduino Bootloader

- The bootloader is a small program stored at the **top of Flash memory**. After reset, the AVR actually starts executing from the bootloader because of the configured **BOOTRST fuse**. The bootloader waits briefly for a new sketch over the USB serial connection and, if none arrives, jumps to the user program.
- Since you programmed the **ATmega328P** using **USBasp**, you may have overwritten the Arduino bootloader. This is normal because **USBasp writes directly to Flash memory**, including the bootloader area if instructed to.
- The Arduino Uno R3 normally contains the **Optiboot** bootloader.

Arduino Bootloader Cont.

- The ATmega328P has **32 KB of Flash memory**. A small section at the **top of Flash** is reserved for the bootloader.
- Earlier Arduino boards used larger bootloaders (around 2 KB), while Optiboot reduced this to about 512 bytes, leaving more Flash available for sketches.
- Fuse bits and lock bits configure the microcontroller so that the bootloader section is protected from accidental overwriting during normal uploads.



Restoring the Arduino Bootloader

- Connect **USBasp** to the Arduino Uno ICSP header.
- Open Arduino IDE.
 - Select: Tools → Board → Arduino Uno
 - Select: Tools → Programmer → USBasp
 - Click: Tools → Burn Bootloader
 - Note: No need to select the port. Ignore the error.